Dossier d’architecture technique SESAM

Architecture web en n-tiers conteneurisée

Version V.04 · 4 novembre 2026

Table des matières

1 Objet et périmètre 2

2 Architecture générale et échanges 3

3 Déploiement sur Docker 4

4 Interfaces et protocoles utilisés 5

5 Justification des choix techniques 6

6 Versions du socle technique 7

7 Environnements et exploitation 8

8 Hébergement sur Internet 9

1 Objet et périmètre

SESAM repose sur une architecture en n-tiers : une interface React, une application backend Spring Boot unique et une base PostgreSQL. L’interface couvre notamment la gestion des bâtiments, des salles, des matériels et des comptes. Les échanges métier utilisent une API REST et des données JSON.

Ce document décrit les responsabilités des composants, leurs échanges, les déploiements Docker de développement et de production et les dispositions d’exploitation. Il présente séparément les adaptations nécessaires à l’hébergement cible C1NP. Les deux schémas présentent l’architecture générale et le déploiement de production.

Les références CCT renvoient au Cadre de cohérence technique du ministère des Armées 2023-2024, version 3.5, édition 2. Les numéros de page correspondent à cette édition. L’applicabilité des versions logicielles et des exigences devra être confirmée avec la gouvernance technique pour l’hébergement cible.

2 Architecture générale et échanges

Le frontend assure la présentation et les interactions avec l'utilisateur. Le backend expose les services métier, contrôle les droits, valide les données et gère leur persistance. PostgreSQL conserve les données relationnelles ; les accès applicatifs à la base passent exclusivement par le backend.

Le serveur de l'interface et l'application React ont des rôles distincts. En production, Nginx distribue les fichiers statiques générés par le build React et Vite. Le navigateur charge ces fichiers puis exécute l'interface React. Les appels à l'API sont envoyés à la même adresse publique ; Nginx transmet les requêtes dont le chemin commence par /api/ au backend Spring Boot sur le réseau Docker. Les réponses du backend reviennent au navigateur par Nginx.

En développement, Node.js exécute Vite pour servir l'interface et actualiser les modifications. Dans cette configuration, le navigateur appelle directement le backend selon l'URL configurée.

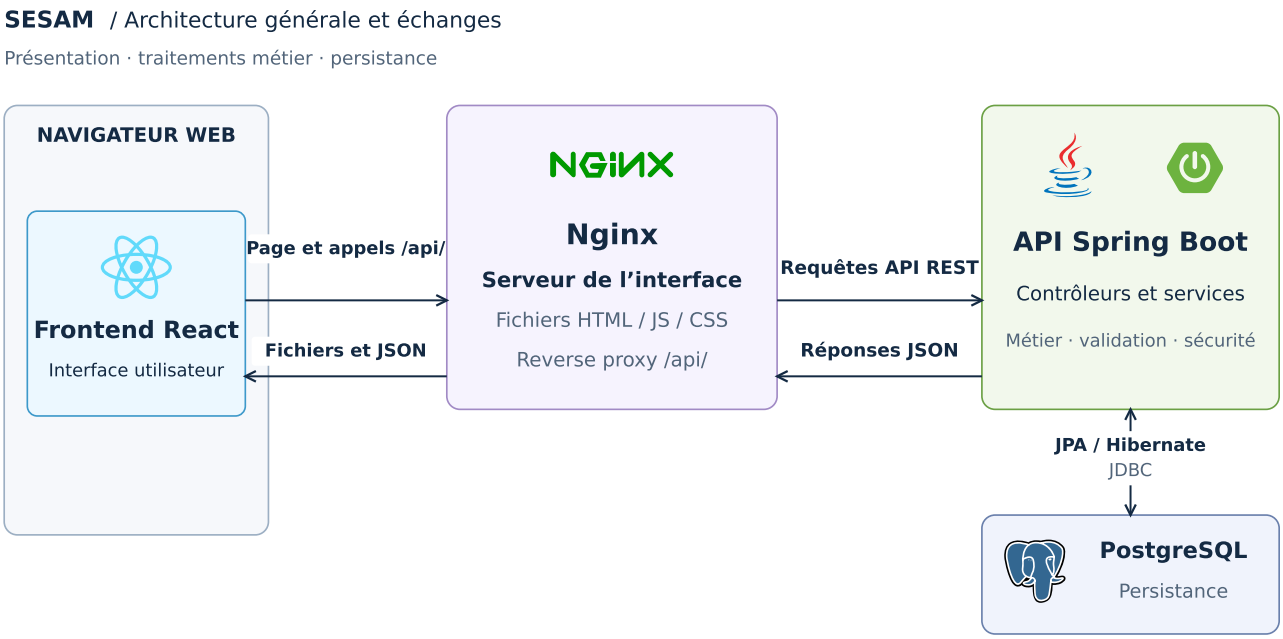


Figure 1 Architecture générale et échanges en production

Échange	Contenu et responsabilité
Navigateur et Nginx	Chargement de la page et des fichiers JavaScript, CSS et autres ressources de l'interface React.
Application React et backend, via Nginx	Fetch envoie les requêtes métier à l'adresse publique. Nginx transmet les appels /api/ au backend, qui traite les demandes et renvoie les réponses JSON.
Backend et PostgreSQL	Spring Data JPA et Hibernate assurent la persistance. Le pilote PostgreSQL établit la connexion JDBC.

Spring Security protège les routes de l'API et contrôle les autorisations. L'authentification applicative utilise actuellement des JWT. L'intégration de l'authentification cible MindefConnect Internet est traitée dans la section 8.

3 Déploiement sur Docker

Le déploiement de production repose sur trois conteneurs : front, backend et db. Docker Compose définit leur construction, leur configuration et leur réseau de communication.

Conteneur front. La construction de l'image comprend une étape utilisant Node.js 22 et Vite pour générer les fichiers statiques de l'interface. Ces fichiers sont ensuite intégrés à une image nginx:1.26-alpine. Nginx les distribue sur le port 80 du conteneur et transmet les requêtes /api/ au service backend sur le port interne 8080. Le port publié sur l'hôte est configurable par WEB_PORT et vaut 80 par défaut.

Conteneur backend. Maven 3.9.9 et le JDK Eclipse Temurin 21 sont utilisés pour compiler l'application et produire son fichier JAR. Le conteneur final utilise l'image eclipse-temurin:21-jre et lance l'application avec la commande java -jar app.jar, sous un utilisateur système dédié. Le backend expose son API sur le port 8080 du réseau Docker. Ce port n'est pas publié sur l'hôte en production.

Conteneur db. L'image postgres:17-alpine exécute PostgreSQL. Le backend la contacte par le nom de service db sur le port interne 5432. Aucun port de ce service n'est publié sur l'hôte. Le volume postgres_data conserve les données de la base.

Volume de production	Usage
postgres_data	Conservation des données PostgreSQL.

La production utilise uniquement le volume postgres_data. Les fichiers statiques du frontend et le JAR du backend sont intégrés aux images finales. Aucun dossier source n'est monté dans les conteneurs de production.

Le fichier compose.override.yaml définit les adaptations de développement. Il rétablit Vite, le lancement du backend avec Maven et entr, les montages des dossiers ./Front et ./Backend, ainsi que les volumes maven_cache et front_node_modules. Il publie également les ports nécessaires aux accès directs depuis le navigateur.

Le backend attend que le contrôle de santé PostgreSQL soit positif avant de démarrer. Le frontend dépend du démarrage du backend. Les services de production disposent de la politique de redémarrage unless-stopped.

Schéma de déploiement

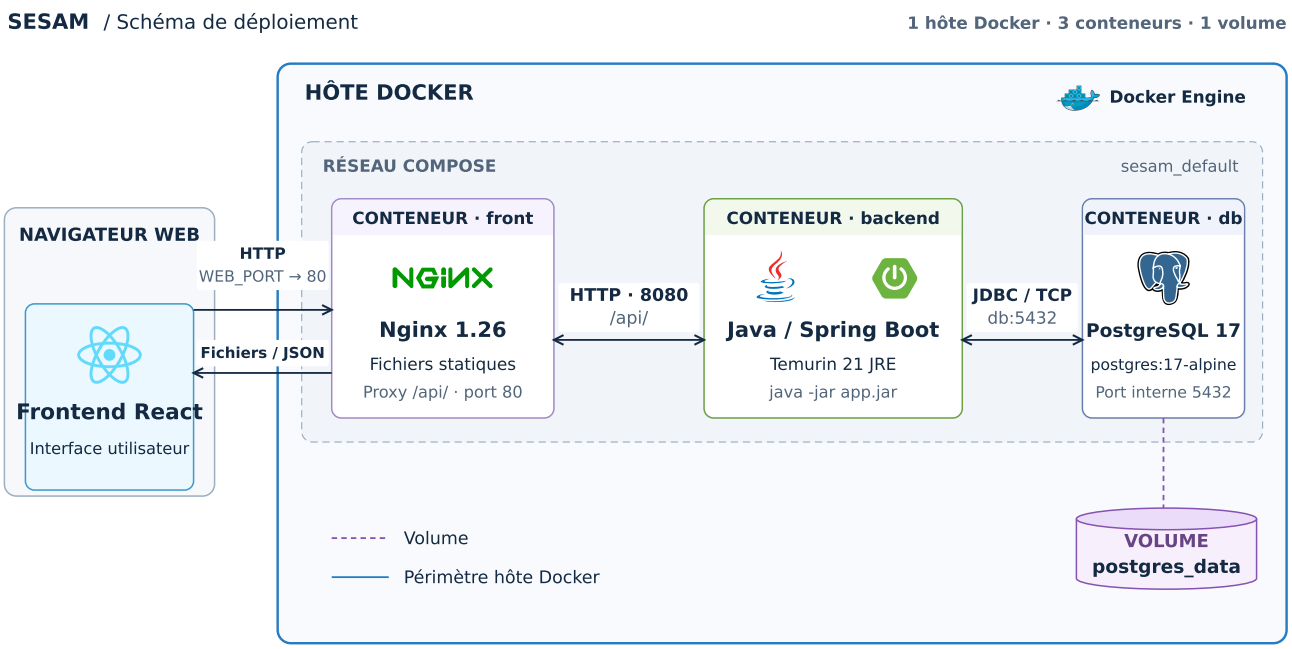


Figure 2 Déploiement de production sur un hôte Docker

4 Interfaces et protocoles utilisés

En production, le navigateur utilise une adresse publique unique pour charger l'interface et appeler l'API. Nginx reçoit les requêtes HTTP, distribue les fichiers du frontend et transmet les appels /api/ au backend. Le backend communique ensuite avec PostgreSQL sur le réseau Docker.

Échange	Port	Protocole	Rôle
Navigateur vers Nginx	WEB_PORT sur l'hôte, 80 par défaut, vers le port 80 du conteneur	HTTP	Chargement de l'interface et réception des appels API.
Nginx vers backend	backend:8080 sur le réseau Docker	HTTP / REST / JSON	Transmission des requêtes /api/ et des réponses métier.
Backend vers PostgreSQL	db:5432 sur le réseau Docker	JDBC / protocole PostgreSQL sur TCP	Lecture et enregistrement des données.

Seul le port du frontend est publié sur l'hôte en production. Le backend et PostgreSQL sont accessibles par les autres services du réseau Docker.

La configuration Nginx actuelle écoute en HTTP sur le port 80. Elle ne configure pas de terminaison HTTPS ; les modalités d'exposition HTTPS de l'hébergement cible restent à définir avec l'hébergeur.

5 Justification des choix techniques

Les technologies retenues correspondent aux compétences maîtrisées par l'équipe de développement. Ce choix réduit le temps de prise en main, facilite la maintenance et limite les risques d'erreurs pendant le développement. Il rejoint le principe de rationalisation des techniques et des compétences du CCT (§ 3.2.1.1, p. 60).

La séparation en n-tiers distingue l'interface, les règles métier et la persistance. Elle permet de faire évoluer chaque partie tout en conservant des interfaces explicites. L'interface web évite une installation applicative spécifique sur les postes, en cohérence avec le principe de client léger du CCT (§ 3.2.1.1, p. 60).

React et TypeScript. React permet de composer des vues réutilisables adaptées aux écrans de gestion de SESAM. TypeScript détecte certaines incohérences de types pendant le développement. Le CCT recommande les applications SPA (RT_DEV_10, § 7.3.5, p. 202), référence React comme recommandé (p. 204) et recommande TypeScript (RT_DEV_14, p. 203).

Spring Boot et Java 21. Spring Boot structure l'API REST et réunit les fonctions de validation, de sécurité et d'accès aux données. Le CCT recommande Spring et décrit les apports de Spring Boot (§ 7.3.2, pp. 200-201).

Spring Security et JWT. Spring Security centralise les contrôles d'accès du backend. JJWT permet de créer et de vérifier les jetons utilisés par l'authentification applicative actuelle.

PostgreSQL. Le modèle relationnel convient aux données structurées de SESAM et aux liens entre bâtiments, salles, matériels et utilisateurs. Il permet d'exprimer des contraintes et de conserver des données transactionnelles. Le CCT privilégie PostgreSQL pour les nouveaux développements et les migrations (§ 3.2.5.3.1, pp. 86-87).

RSQL. RSQL permet de construire des recherches combinant plusieurs critères. Son intégration avec Spring Data JPA facilite le filtrage des données et limite la multiplication des méthodes de recherche spécifiques dans le backend.

Node.js et Vite. Node.js exécute les outils de développement et de construction du frontend. Vite fournit le serveur de développement et génère les fichiers statiques destinés à la production. Node.js intervient dans la construction de l'image ; le service frontend de production est exécuté par Nginx. Le CCT restreint Node.js aux versions paires LTS soutenues (§ 3.2.2.2.4, p. 72).

Nginx. Nginx distribue les fichiers statiques de l'interface React et centralise les accès à l'application. Son rôle de reverse proxy permet de transmettre les appels API au backend sans publier directement le port de celui-ci. La configuration prévoit également la compression des réponses et la mise en cache des ressources statiques.

Bootstrap 5. Les composants et styles réutilisables accélèrent la réalisation des écrans et rendent leur présentation homogène. Bootstrap 5 est référencé comme recommandé dans l'annexe des produits du CCT (§ 8.2.1, p. 237).

Docker et Logback. Docker Compose organise les services et permet de reproduire les environnements. Le choix du runtime de production pour l'hébergement cible suit les contraintes OCI du CCT (§ 5.1.3, pp. 160-161). Logback produit les journaux du backend pour leur collecte ultérieure.

6 Versions du socle technique

Les versions ci-dessous sont déclarées dans les fichiers du projet. Pour npm, ^ et ~ définissent des plages de versions ; package-lock.json fixe les versions installées par npm ci. Les tags d'images Docker peuvent évoluer : les versions exactes et les empreintes des images devront être tracées pour chaque livraison.

Technologie	Version déclarée	Fonction
React et React DOM	^19.2.8	Interface et rendu
TypeScript	~6.0.2	Typage statique
Vite	^8.3.0	Serveur de développement et build
React Router	^7.18.4	Navigation entre les vues
Bootstrap	^5.3.8	Composants et styles
Bootstrap Icons	^1.13.1	Icônes
ESLint	^10.10.0	Analyse statique
TypeScript ESLint	^8.69.0	Règles TypeScript
Java	21 / Eclipse Temurin JDK et JRE	Compilation avec le JDK ; exécution en production avec le JRE.
Maven	3.9.9	Construction du backend et lancement en développement.
Spring Boot	4.1.1	Socle backend
JJWT	0.13.0	Gestion des jetons JWT
PostgreSQL	17 / postgres:17-alpine	Persistance relationnelle
Node.js	22 / node:22-alpine	Développement et construction du frontend.
Nginx	1.26 / nginx:1.26-alpine	Distribution du frontend et reverse proxy vers l'API.
Logback	Gérée par les dépendances Spring Boot	Production des journaux du backend
Springdoc OpenAPI	2.8.14	Documentation OpenAPI et consultation interactive avec Scalar.
RSQL - rsql-jpa-spring-boot-starter	7.0.2	Filtres de recherche dynamiques traduits en critères JPA.

Modules backend

Spring Web, Spring Data JPA, Spring Validation, Spring Security et Spring Boot Actuator composent le socle backend. Le pilote PostgreSQL, Lombok et RSQL le complètent. Springdoc OpenAPI 2.8.14 génère la documentation de l'API ; le module springdoc-openapi-starter-webmvc-scalar fournit l'interface de consultation interactive Scalar. Les dépendances jakarta.json-api et Parsson assurent le traitement des documents JSON utilisés pour les modifications partielles au format JSON Merge Patch. Les versions transitives, dont celle de Logback, doivent figurer dans l'inventaire issu de l'arbre Maven et dans la nomenclature logicielle de livraison.

7 Environnements et exploitation

Environnement	Organisation du code	Usage
Développement	Branches dédiées aux fonctionnalités	Développement et vérifications locales.
Production	main	Version destinée à être livrée aux utilisateurs.

Chaque fonctionnalité est développée dans une branche dédiée. Après revue et validation, les changements sont intégrés dans main. Une livraison doit être associée à un commit et à des images identifiés.

Configuration

La configuration de production utilise PUBLIC_URL pour définir l'adresse publique de l'application. Cette valeur est transmise au build du frontend par VITE_API_URL et utilisée par le backend pour configurer FRONT_ORIGIN. WEB_PORT définit le port HTTP publié sur l'hôte. PUBLIC_URL doit correspondre à l'adresse et au port réellement utilisés par les navigateurs.

En production, VITE_API_URL est intégrée aux fichiers du frontend pendant leur construction. Une modification de cette adresse nécessite donc de reconstruire l'image frontend. Les paramètres de connexion PostgreSQL et le secret JWT sont fournis au backend par des variables d'environnement.

Le Compose contient des valeurs de repli de développement. Les secrets réels doivent être fournis par un mécanisme externe adapté à l'environnement, sans intégration dans le code ou les images. Les variables VITE_ exposées au frontend sont publiques et ne doivent contenir aucun secret.

Pour une installation locale utilisant PUBLIC_URL=http://localhost:8081, WEB_PORT doit être défini à 8081. Avec le port public 80 par défaut, l'adresse correspondante est http://localhost. Pour un accès distant, l'adresse doit être joignable depuis les navigateurs.

Procédure de déploiement de production

1. Identifier le commit à installer et construire les images front et backend avec une référence traçable.

2. Préparer PUBLIC_URL, WEB_PORT, les paramètres de connexion et le secret JWT, ainsi que le volume postgres_data.

3. Sauvegarder la base existante avant une évolution du schéma. La configuration actuelle utilise Hibernate ddl-auto: update ; des migrations versionnées doivent être définies pour la production.

4. Démarrer les services en sélectionnant uniquement compose.yaml, puis contrôler la base, le backend et l'accès public par Nginx.

5. Tester la connexion utilisateur et une opération métier complète avant de valider l'installation.

Depuis le dossier app : **docker compose -f compose.yaml up -d --build**

Sans sélection explicite des fichiers, Docker Compose charge aussi compose.override.yaml et réactive la configuration de développement.

8 Hébergement sur Internet

Cette section présente les dispositions prévues pour l'hébergement cible sur Internet. Elles complètent la configuration Docker présentée dans les sections précédentes.

Conformément au point 6.1.6 du Cadre de cohérence technique, relatif à l'hébergement sur Internet, l'hébergement de l'application est prévu sur le réseau C1NP. Ce déploiement implique de disposer d'une pile logicielle à jour et maintenue en condition opérationnelle et de sécurité (MCO/MCS), conformément aux conditions d'homologation déjà applicables aux hébergements actuels (§ 6.1.6.2, p. 184).

Authentification et journaux

Le projet est actuellement en cours d'homologation et l'intégration de MindefConnect Internet reste à réaliser. Pour les besoins de démonstration, le scénario d'authentification sera présenté à l'aide d'une page de connexion standard. Ce dispositif provisoire ne se substitue pas à l'authentification MindefConnect Internet exigée pour l'hébergement cible (§ 6.1.6.2, p. 184).

Un puits de logs sera utilisé afin de centraliser les journaux d'événements applicatifs et de faciliter leur consultation et leur analyse. Logback assure la génération des journaux du backend Spring Boot. Pour C1NP, ces journaux devront être exportés vers le puits fourni par le socle ; les modalités de collecte, de transmission et de conservation seront définies avec l'hébergeur (§ 6.1.6.2, p. 184).

Exécution des conteneurs

Docker reste l'outil du déploiement de développement. L'architecture et la pile logicielle seront soumises au SC²A avec les Dockerfile ou équivalents et les nomenclatures logicielles SBOM, qui recensent les composants et leurs versions.